Block Devices and the Cost of Small Reads

R. Ackon · Beneath the Pipeline

A storage device presents one interface to the software above it: a long row of numbered blocks, each of a fixed size. A common size is 4096 bytes. The device will not hand back less than a block, and it will not write less than a block. Every abstraction above it — files, folders, databases, indexes — is built out of that one operation.

The consequence shows up immediately in measurement. Reading ten thousand small files costs far more than reading one file of the same total size, because each file carries a fixed overhead that has nothing to do with how many bytes it holds. The overhead is per request, not per byte, so a workload made of many small requests pays it many times.

Storage engines are largely a response to this fact. They batch writes so that many logical updates become one physical transfer. They keep their own bookkeeping instead of using the filesystem's, because the filesystem's bookkeeping is optimised for a different access pattern. They arrange records so that a query touches contiguous blocks rather than scattered ones.

Measured cost of four read strategies

strategy	requests	bytes moved	relative cost
file at a time	10,000	41.2 MB	14.2
line at a time	4,118	41.2 MB	3.2
one buffered read	37	41.2 MB	1.0
memory mapped	12	41.2 MB	0.9

The same reasoning governs index construction. An index is an arrangement chosen so that a question becomes cheap to answer. The work does not disappear; it moves from query time to build time, and is paid for in memory, in build duration, and in the cost of keeping the arrangement current as data changes.

None of this is visible from the top of the stack. A call that loads a document returns a string, and the string does not record how many requests crossed into the operating system to produce it, nor how many blocks were read and discarded on the way.